# Edge-Healer: Decentralising Kubernetes with Edge Self-Healing

Davide Lamagna
*Universitat Politecnica de Catalunya, Barcelona, Spain*
*davide.lamagna@estudiantat.upc.edu*

***Abstract*—Traditional Kubernetes relies on a highly consistent, centrally hosted control-plane, an assumption that breaks down in thin-WAN or intermittently connected edge deployments. When the API-server is unreachable, lost Pods remain unreplaced and production lines stall. We present *edge-healer*, a lightweight self-healing framework that decentralises recovery logic by running a Serf-backed gossip sidecar and a Python (soon Go) bidding agent as a DaemonSet on every worker node. Nodes periodically exchange free-CPU budgets and, upon detecting a missing Pod, issue a bind decision to recreate it on the best-suited peer—without waiting for the control plane's scheduler or ReplicaSet controllers. The prototype is publicly available and shows that modest cluster-side changes can help achieve true edge autonomy while remaining API-compatible with upstream Kubernetes.**

## I. Introduction

Kubernetes, originally developed by Google, is an open-source platform that orchestrates, schedules, and scales containerised applications. Thanks to its abilities of efficiently deploying and managing thousands of interconnected services, it has become widely adopted within cloud-native development [2]. Organisations leveraging cloud-native technologies frequently use Kubernetes to dynamically scale applications, manage intense workloads, and deliver services efficiently. In modern day applications, such as Uber, [3], it is not abnormal for an application to contain more than a thousand microservices, with throughput several magnitudes higher.

Kubernetes has become the industry standard for container orchestration, and yet, it has a weakness when deployed to IoT or edge environments by traditionally following a master-worker model with a centralised control plane. The reason for doing this is straightforward: Kubernetes' aim is to abstract away complicated details of container scheduling, networking, and scaling, and facilitate ease of management for complicated cloud networks. Its design follows principles of high consistency, where you know that what you are seeing is correct as a matter of fact, at the price of availability and fault tolerance.

The master (control plane) makes all global decisions for the cluster, such as scheduling Pods or handling state changes, while worker nodes simply execute the workloads. This centralisation simplifies design but can introduce single points of failure and scaling bottlenecks. The question arises: *how can the Kubernetes architecture be extended so that "agents manage themselves" rather than relying on a single centralised brain?*

An outcome was reached with the *edge-healer* pod, written in Python and packaged in docker, and deployed on Kubernetes worker nodes. The edge-healer pod contains a bidding logic, which is called by the kubelet when a pod dies and binds the pod to the best available node. Information of free CPU and the best available node is shared between the pods with serf-based gossip in Go binary sidecars.

The rest of the paper is organised as follows: The paper continues by explaining the key concepts of Kubernetes' control plane, the gossip protocol, and self-healing. Existing solutions are discussed, such as industry standard KubeEdge and KubeFed, as well as research already completed on this topic. The explored solution is then explained, as well as the evaluation of the solution and items of future development.

## II. Background

### A. Kubernetes: Limitations of the control plane

The control plane in Kubernetes acts as the "brain" of the cluster, and contains the below components [4], as seen in figure 1:

- **kube-controller-manager:** Runs control loops that monitor cluster state and drive it toward the desired state (e.g., managing replicas or node health). In production environments, all core controllers are compiled into a single binary that uses leader election between master nodes for high availability (HA Kubernetes).
- **kube-api-server:** Acts as the central gateway for all cluster operations, exposing a RESTful API interface. It handles authentication, validation, and concurrency, and mediates all access to the cluster's state stored in etcd.
- **etcd:** A strongly consistent key-value store that holds the entire cluster state, including workloads and configurations. It is implemented using the RAFT consensus algorithm and serves as the single source of truth for the system. Consensus is reached by a quorum of 3-7 nodes.
- **kube-scheduler:** Responsible for assigning Pods to nodes based on resource constraints, affinities, and workload requirements. It operates as a centralized process with a global view of the cluster and places Pods sequentially.

Some additional Kubernetes-specific vocabulary and concepts which are referred to throughout the paper are listed below:

- **Replicas and ReplicaSet:** A ReplicaSet is a control loop that ensures a specified number of identical Pods (Replicas) are running at all times. It automatically creates or

deletes Pods to match the desired replica count, serving as the underlying mechanism behind higher-level controllers like Deployments.

- **Bind vs Create:** Pod creation (Create) registers the Pod object in the API server, placing it in a Pending state. The scheduler then performs a Bind operation, assigning the Pod to a specific node; binding is a distinct step that can be customised or intercepted by external schedulers.
- **Kubelet:** The kubelet is a node-local agent that watches the API server for assigned Pods and ensures their containers are running according to specification. It interacts with the container runtime (e.g., containerd/docker) and reports node and Pod health back to the control plane. The kubelet knows the state of pods within its own node-space, including if a container has died.
- **DaemonSet:** A DaemonSet ensures that a copy of a specified Pod runs on every node (or a subset defined by label selectors). It is commonly used for deploying node-local services such as log collectors, network agents, or monitoring daemons, and in this paper it is used to ensure an instance of the edge-healer Pod is run on each node.
- **Pod Lifecycle:** A Pod's lifecycle begins with its creation and includes phases such as Pending, Running, Succeeded, Failed, and Unknown. Transitions between phases are driven by scheduling, container runtime states, and failure conditions; Kubernetes does not restart a Pod once it has terminated — a controller must explicitly create a replacement.

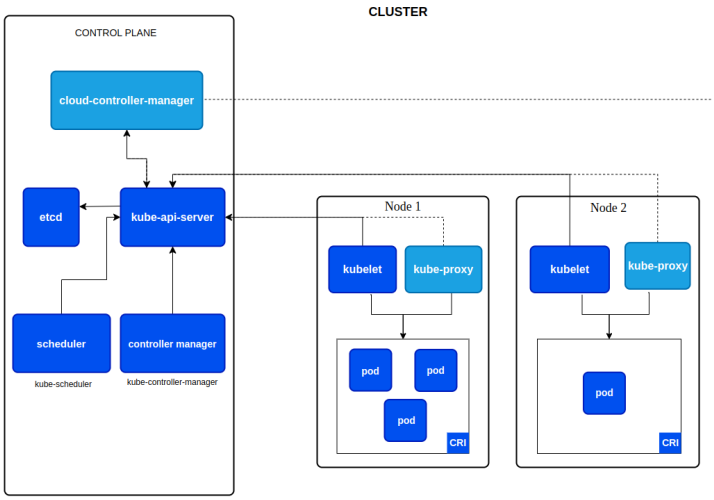



Fig. 1. The basic components within a Kubernetes cluster [4]

Kubernetes' single API-server and scheduler control-plane is intentionally centralised to simplify control, scheduling and policy enforcement. However, in times when the API server is not reachable, there is a risk of a "freeze-brain" scenario, where worker nodes are stuck waiting for decisions from the control plane.

For example, when considering an IoT-edge deployment at an industrial factory: Factory gateways and on-prem devices connect to a remote cloud region over a thin WAN link that could potentially drop for several minutes per day. During such outages, any node crash or power-cycle leaves Pods permanently "lost" because the ReplicaSet and scheduler loops that would re-create them are stranded behind the unreachable control-plane. For a production line whose SLAs cannot tolerate gaps longer than a few seconds, this single-point dependency is a problem.

### B. Self-healing

In Kubernetes' default architecture, self-healing is orchestrated centrally: controllers such as the ReplicaSet controller which monitors the number of Replicas continuously monitor the desired versus actual state of the cluster, and automatically re-create Pods when they crash or are deleted [5]. This behavior, often cited as a key strength of Kubernetes, ensures that workloads remain available without manual intervention.

More broadly, self-healing in distributed systems refers to the system's ability to detect, diagnose, and recover from faults autonomously, minimizing human involvement. Ideally, this would include localised decision-making and autonomous recovery, particularly in large-scale or edge deployments where centralised coordination may be unavailable or too slow. This is where vanilla Kubernetes falls short: use cases such as industrial IoT, autonomous vehicles, or remote monitoring have a fundamental mismatch. The orchestration logic remains centralised, even as the workloads are increasingly distributed. Enabling genuine self-healing in such contexts requires shifting recovery logic closer to the edge, allowing nodes or lightweight agents to make recovery decisions without waiting for a response from the distant control plane.

### C. Gossip Protocols and SWIM

Gossip protocols are decentralized communication mechanisms that rely on periodic, randomized peer-to-peer exchanges to share state information across a distributed system. In contrast to traditional centralized coordination, gossip protocols ensure eventual consistency by having each node share updates with a small, randomly selected subset of peers at regular intervals. This approach is by default resilient to failure and scales efficiently in large clusters.

The SWIM protocol [7] (Scalable Weakly-consistent Infection-style Membership) is a foundational work in this space, proposing a two-part mechanism: a randomized probe-based failure detector and a "piggybacking" dissemination component. Each node periodically selects a random peer to probe; if the peer does not respond within a timeout, indirect probes are used to confirm the failure before broadcasting it to others. Updates about membership and node liveness are piggybacked onto probe messages, allowing state to travel through the system with minimal additional overhead.

Mathematically, the dissemination process achieves *logarithmic convergence*: a membership update reaches all nodes with high probability in $O(\log n)$ time, where $n$ is the number of nodes. Additionally, each node communicates with only a constant number of peers per round, resulting in $O(n)$ total

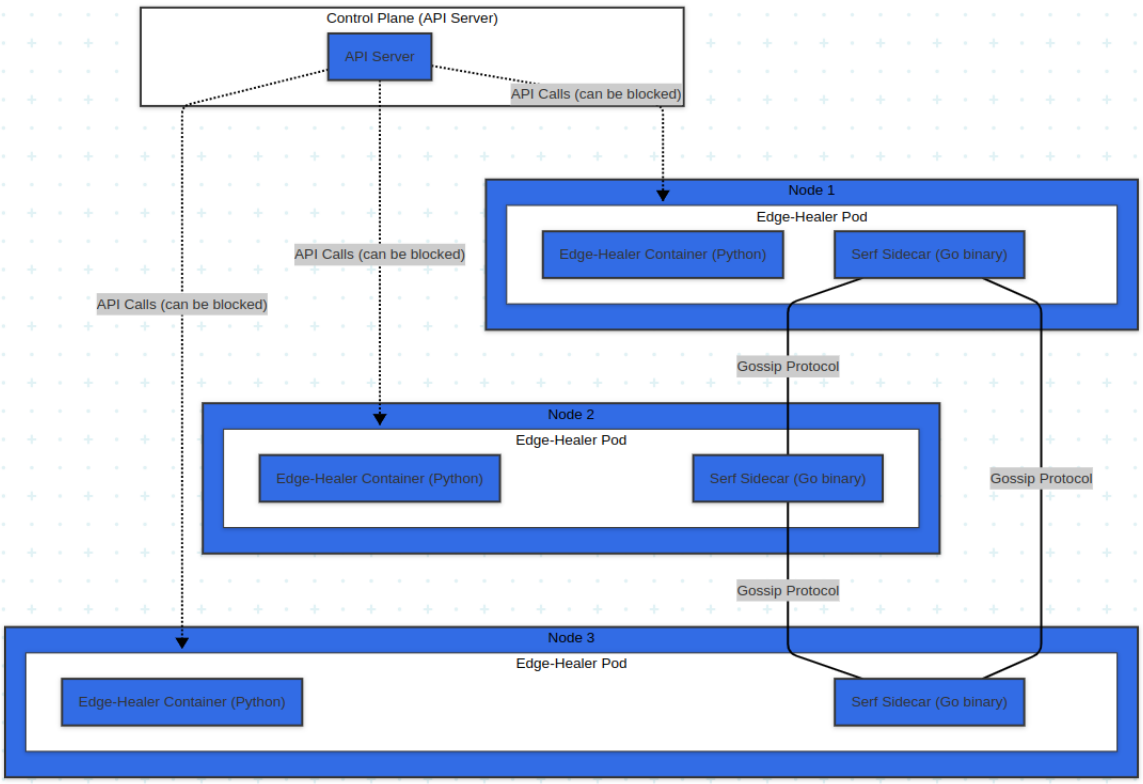



Fig. 2. Our implemented 3 worker node Kubernetes cluster, with gossip between the serf sidecars [1]

message load per round across the system. This is a key factor in avoiding broadcast storms or communication bottlenecks.

This epidemic-style propagation and constant per-node overhead make gossip protocols particularly suitable for horizontally scalable environments like edge clusters, where central coordination may be unavailable or too expensive. For this reason, systems such as *Serf* and *memberlist* adopt the SWIM approach to maintain group membership and enable decentralised self-healing without incurring global coordination costs.

In this work, we adopt gossip-based communication not for failure detection, but for decentralised resource sharing: nodes broadcast their free CPU availability to peers in a lightweight and fault-tolerant manner. The implementation leverages *Serf*, a production-grade library based on SWIM, which simplifies group membership and gossip messaging while abstracting away low-level protocol details.

### *D. KubeFed, KubeEdge, and previous research*

When a single cluster is not enough, **KubeFed** (Kubernetes Federation) lets operators treat many clusters as one logical fleet. A central federation control plane observes member clusters and propagates desired resources; each local scheduler then performs fine-grained placement. This model is popular for *hyper-mesh* deployments—global SaaS or telco platforms that must stamp identical workloads onto dozens or hundreds of regional clusters while keeping lifecycle, quotas and policy consistent [8].

Complementing that, **KubeEdge** pushes part of the control-plane logic onto the node itself. Its *CloudCore* component syncs only the necessary metadata to an *EdgeCore* agent (deployed on every node) that continues to run workloads, manage devices and resolve Services even when the cloud link is down. That edge-cloud split directly targets IoT and smart-factory scenarios where thousands of sensors or cameras generate data that must be processed locally yet still be orchestrated via familiar Kubernetes APIs.

Outside of official Kubernetes add-ons, there is other literature which is relevant to this work. Early attempts to decentralise placement logic draw inspiration from biological swarms: Rusek and Dwornicki migrate containers between hosts using “pheromone” agents to balance load without a central scheduler [6], which conceptually, is close to our bidding strategy. Previous work of implementing serf into Kubernetes [15] can cut fail-over detection from seconds to sub-second ranges, yet still leaves replica recreation to the cloud control-plane [14].

**Liqo** federates clusters so that workloads transparently roam across sites, while **Medik8s** and **Active-Monitor** focus on node-level remediation and workflow-driven healing, respectively [11]–[13]. These tools improve availability but do not coordinate replica re-scheduling once the WAN is severed.

Recent research heads toward AI-assisted resilience. Graph-based anomaly detection for Kubernetes [16] and energy-aware self-healing at the edge [17] show promising accuracy,

yet they monitor clusters that still presume a live control-plane. In contrast, our work targets the moment that link disappears: a co-operative Serf-gossip ring that cache-boots desired state and races to rebind lost Pods, closing the gap left by existing approaches.

## III. Edge-Healer Solution

### A. Cluster Changes: Edge-Healer Architecture

To enhance resilience in edge Kubernetes clusters, we introduce a lightweight self-healing agent named *edge-healer* [1]. This agent is deployed as a *DaemonSet*, ensuring that each node in the cluster runs an instance. The primary components and functionalities include:

- **Serf Sidecar Integration**: Each *edge-healer* pod includes a Serf agent as a sidecar container. This setup facilitates decentralized communication among nodes using a gossip protocol. Nodes periodically exchange health status and available CPU capacity, enabling the cluster to make informed decisions without relying on the central control plane.
- **Kopf Operator**: Leveraging the Kopf (Kubernetes Operator Pythonic Framework), the agent monitors *Pod* and *ReplicaSet* resources. This monitoring allows the agent to detect anomalies, such as missing pods, and initiate corrective actions.
- **Optimistic Rebinding**: In scenarios where the control plane becomes unreachable, the agent can autonomously recreate missing pods on the most suitable node based on available resources. This rebinding process is designed to be swift, aiming to complete in under a second (industry standard is 2 seconds).
- **State Reconciliation**: Upon restoration of the control plane, the agent reconciles the cluster state to prevent issues like split-brain scenarios, ensuring consistency across the cluster.

This architecture, illustrated in figure 2 ensures that edge nodes can maintain operational continuity even during control plane outages, enhancing the robustness of the edge computing environment.

The code is currently publicly available and for experimentation and further research, and interested readers are encouraged to contribute, make comments, and submit contributions.

### B. Differences to KubeEdge

KubeEdge extends Kubernetes to the edge with a hierarchical architecture: each node runs EdgeCore for local runtime and device management, but a cloud-resident CloudCore remains the single source-of-truth. Edge nodes cache objects they previously pulled from CloudCore and can keep those Pods running while the WAN is down, yet all life-cycle actions that require new API objects (e.g. binding a replacement Pod, scaling a Deployment, enforcing HPA) must wait until connectivity is restored. Failure signals are relayed upward to CloudCore, and reconciliation happens only after the edge re-establishes contact, so a lost node or Pod can remain unrecovered for minutes or hours.

On the other hand, the proposed self-healer DaemonSet removes that bottleneck by giving every edge node an equal role in control decisions the moment the WAN cuts off. A tiny Go agent, deployed as a different DaemonSet, watches the subset of workloads labelled for edge autonomy and exchanges a lightweight gossip message containing its free-CPU budget. When a node or Pod disappears, the edge-healer agent runs a local bidding routine: the first node with sufficient spare CPU issues an optimistic `PATCH /bind` that recreates the missing Pod on site, with no cloud master involved. Desired state manifests are cached in an embedded BoltDB/Badger store so that even a cold reboot during WAN outage can repopulate all workloads. When connectivity returns, the agents push their locally created objects back to the central API, relying on versioned writes to avoid duplicates. By consequence, the design delivers true self-healing autonomy and scales control capacity horizontally every new node adds not just compute for applications but also scheduling and controller capability without increasing load on a distant CloudCore.

## IV. Evaluation and Results

### A. Experimental setup

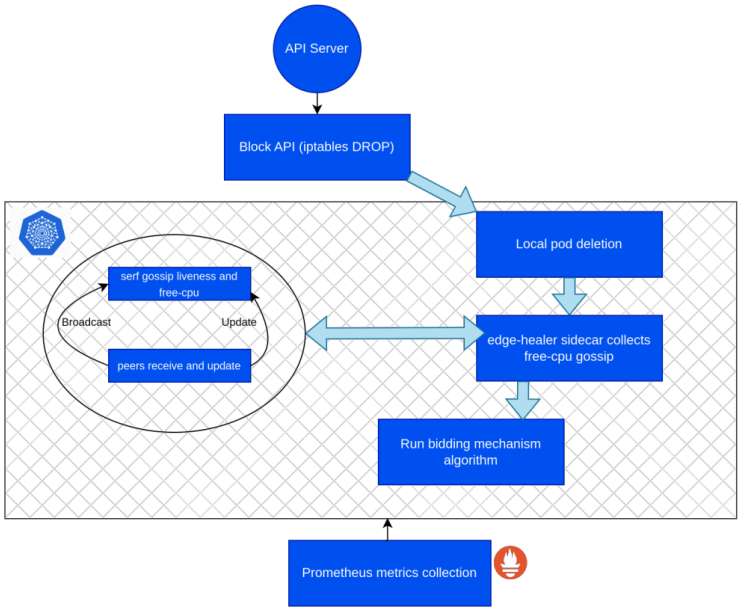



Fig. 3. Visual workflow of our experimental setup (more detail included in the github README [1])

To validate the effectiveness of the *edge-healer* solution, we conducted experiments using a simulated edge cluster environment. The setup details are as follows:

- **Cluster Configuration**: A four-node Kubernetes cluster was deployed, with each node running the *edge-healer* DaemonSet. The nodes were configured to simulate edge devices with intermittent connectivity to the central control plane (where we cut off connection to the API manually).

- **Workload Deployment**: Sample workloads (referred to as *BusyBox*) were deployed across the cluster to emulate real-world applications. These workloads were intentionally disrupted to test the self-healing capabilities of the system.
- **Failure Simulation**: Network partitions and control plane outages were introduced to assess the agent's response. Metrics such as pod recovery time and resource redistribution were recorded.
- **Monitoring and Logging**: Comprehensive logging was enabled to capture the agent's actions during failure scenarios. This data provided insights into the decision-making process and effectiveness of the self-healing mechanisms.

Initial validation showed successful pod restarts in times of disconnect to the control plane, with logs recording successful identification, gossip, optimistic binding, and reconciliation once the connection to the control plane was reestablished.

### B. Future development

To further evolve and validate the proposed *edge-healer* design, we outline several potential avenues for future research and engineering development:

- **Implementation in Go:** Rewriting the agent in Go would enable tighter integration with the Kubernetes API machinery and reduce runtime overhead. It would also make the project more aligned with the broader Kubernetes ecosystem and improve adoption potential among operators.
- **Lightweight Execution on IoT-Scale Nodes:** A core goal is to demonstrate self-healing on truly constrained hardware, such as worker nodes with less than 30 MB RAM. This will require optimizing the binary footprint and run-time efficiency, which is more feasible in Go than in Python.
- **Larger-Scale Deployment:** Testing the system in a cluster with dozens or hundreds of nodes would help evaluate scalability, especially the efficiency of gossip dissemination and the performance of the optimistic rebinding mechanism under high cpu stress.
- **Stress Testing WAN Partitions:** One key question is how long the control plane can remain unreachable before the decentralized coordination begins to diverge from the global state, given that we are binding optimistically when the API is unreachable. Measuring this threshold would also inform best practices for edge deployment strategies.
- **Comparison with KubeEdge:** While KubeEdge implements an "offline mode" using state caching and device twin synchronization, it still assumes a hub-and-spoke model. A comparative evaluation could reveal trade-offs in latency, correctness, and resource utilization between centralized caching and decentralized self-healing.
- **Open Source Helm Chart:** Packaging the project as a Helm chart would facilitate broader community adoption and reproducibility.

## V. Conclusion

This work highlights a fundamental conflict in edge deployments: Kubernetes' centralised control-plane guarantees strong consistency but sacrifices availability when the WAN link fails. By embedding a Serf gossip mesh and a cooperative bidding agent in every node, *edge-healer* shifts core self-healing decisions to the data-plane itself. Experiments on a simulated cluster show that the approach restores workloads successfully and has the potential of doing so within stringent SLA windows, only with a constant, low message cost per node.

Beyond raw recovery speed, the solution preserves the Kubernetes object model, caching manifests locally and reconciling them once connectivity resumes, thereby avoiding "split-brain" scenarios. Nevertheless, limitations remain: the current agent is written in Python and assumes moderate node resources, and large-scale behaviour under hundreds of nodes and prolonged partitions still needs validation. Nevertheless, the implementation substantiates that lightweight, gossip-driven autonomy is a viable path toward resilient edge orchestration. Ongoing work will re-implement the agent in Go, and reduce its memory footprint to sub-30 MB for IoT gateways.

## References


[1] D. Lamagna, "kubernetes-edge-healer: Prototype for decentralized gossip-based edge healing in a 4-node cluster," GitHub repository, [Online]. Available: https://github.com/dlamagna/kubernetes-edge-healer. [Accessed: May 28, 2025].

[2] N. Solodkov and L. Bollmann-Dodd, "State of Cloud Native Development Q1 2025," *Cloud Native Computing Foundation*, Mar. 2025. [Online]. Available: https://www.cncf.io/wp-content/uploads/2025/04/Blue-DN29-State-of-Cloud-Native-Development.pdf

[3] M. Chabbi, C. Zhang, and M. K. Ramanathan, "CRISP: Critical Path Analysis for Microservice Architectures," *Uber Engineering Blog*, Nov. 2021. [Online]. Available: https://www.uber.com/en-ES/blog/crisp-critical-path-analysis-for-microservice-architectures/

[4] Kubernetes Authors, "Components of Kubernetes," *Kubernetes Documentation*, [Online]. Available: https://kubernetes.io/docs/concepts/overview/components/. [Accessed: May 28, 2025].

[5] Kubernetes Authors, "Kubernetes Self-Healing" *Kubernetes Documentation*, [Online]. Available: https://kubernetes.io/docs/concepts/architecture/self-healing/. [Accessed: May 28, 2025].

[6] M. Rusek and G. Dwornicki, "Swarm-Like Distributed Algorithm of Scheduling a Microservice-Based Application to the Cloud Servers," *Electronics*, vol. 10, no. 13, p. 1553, Jul. 2021. DOI: 10.3390/electronics10131553.

[7] A. Das, I. Gupta, and A. Motivala, "SWIM: Scalable Weakly-Consistent Infection-Style Process Group Membership Protocol," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, Washington, DC, USA, Jun. 2002, pp. 303–312.

[8] Tigera, "Kubernetes Federation: Mastering Multi-Cluster Management," *Tigera Guides*, [Online]. Available: https://www.tigera.io/learn/guides/kubernetes-security/kubernetes-federation/. [Accessed: May 28, 2025].

[9] KubeEdge Authors, "EdgeSite: Stand-Alone Control-Plane at the Edge," 2025. [Online]. Available: https://kubeedge-docs.readthedocs.io/en/latest/modules/edgesite.html

[10] KubeEdge Authors, "Edge Autonomy Feature Documentation," 2025. [Online]. Available: https://github.com/kubeedge/kubeedge/tree/master

[11] LiqoTech, "Liqo: Enable dynamic and seamless Kubernetes multi-cluster topologies," *GitHub Repository*, [Online]. Available: https://github.com/liqotech/liqo. [Accessed: May 28, 2025].

[12] Medik8s Community, “Self-Node-Remediation (Medik8s) GitHub Repository,” 2025. [Online]. Available: https://github.com/medik8s/self-node-remediation
[13] Keikoproj Team, “Active-Monitor: Deep Monitoring and Self-Healing of Kubernetes Clusters,” 2025. [Online]. Available: https://github.com/keikoproj/active-monitor
[14] A. Siderova, “Improving the Failure Detection Time of Distributed Mission-Critical Systems,” Master’s thesis, Faculty of Electrical Engineering, Mathematics and Computer Science, University of Twente, Enschede, The Netherlands, 2025. [Online]. Available: https://essay.utwente.nl/106184/
[15] Slack Engineering, “kubernetes-serf Experiment Repository,” 2025. [Online]. Available: https://github.com/slack/kubernetes-serf
[16] N. Author and P. Collaborator, “Enhancing Kubernetes Resilience through Anomaly Detection and Healing,” *arXiv preprint* arXiv:2503.14114, Mar. 2025.
[17] Q. Researcher and L. Analyst, “AI-Driven Self-Healing Container Orchestration for Edge,” *J. Emerging Edge Computing*, vol. 3, no. 2, pp. 45–60, 2025.
[18] KubeEdge Authors, “KubeEdge Documentation Home,” 2025. [Online]. Available: https://kubeedge.io